Federal University of Science and Inovation of Minas Gerais
Research Report



ULP Transitions, Rounding, and Absorption in $(x^{10} + c) - x^{10}$: A Study of IEEE 754 Binary32 and Binary64

Author
Enzo Rocha Leite Diniz Ribas

Supervisor
Lucas Martins Rocha

Coorientador
Luis Alberto D'Afonseca

Belo Horizonte
November 6, 2026

Abstract

In exact arithmetic, the expression $(x^{10} + c) - x^{10}$ is identically equal to c. In finite-precision binary arithmetic, however, its computed value displays a sequence of plateaus, parity-dependent rounding ties, and eventually complete absorption of the increment. This report investigates that behavior in IEEE 754 `binary32` and `binary64`, using the supplied research manuscript, extended book draft, experimental report, notebook, and interactive graphics. Writing $y = \mathrm{fl}(x^{10})$, $u = \mathrm{ulp}(y)$, and $c = m2^{\nu}$ for odd m, the nearest-grid model predicts an initial inexact binade at $E_1 = p + \nu$ and the last potentially nonzero binade at $E_2 = p + \lfloor \log_2 c \rfloor$, where p is significand precision. Consequently, the predicted inexact interval in x is approximately $[2^{(p+\nu)/10}, 2^{(p+1+\lfloor \log_2 c \rfloor)/10})$ and spans $L = \lfloor \log_2 c \rfloor - \nu + 1$ binades. The model is established under explicit assumptions about rounding, reuse of the computed power, and binade boundaries. Results reported in the supplied study cover 1,611 values of c in each precision and distinguish powers of two, odd integers, dyadic fractions, and non-dyadic decimal or reciprocal inputs. Reconstructed visualizations and an independently checked 780-case power-of-two grid illustrate when increments are preserved, rounded, or absorbed. Special attention is given to boundary effects, input representation, the relation to Fast2Sum, and limitations of the model.

Keywords: IEEE 754; floating-point arithmetic; unit in the last place; rounding to nearest; cancellation; absorption; binary32; binary64; numerical reproducibility.

Contents

1 Introduction **7**
- 1.1 Motivation . . . 7
- 1.2 Research question and aims . . . 7
- 1.3 Contributions and provenance . . . 7
- 1.4 Scope and assumptions . . . 8
- 1.5 Organization . . . 8

2 Theoretical Framework **9**
- 2.1 Binary floating-point spacing . . . 9
- 2.2 The computed function and exact final subtraction . . . 10
- 2.3 Nearest-grid rule and ties . . . 10
- 2.4 Thresholds and the width of the inexact region . . . 11
- 2.5 Error measured in ULPs . . . 11
- 2.6 Qualifications at binade borders and for $y < c$ . . . 11

3 Experimental Design and Reproduction **13**
- 3.1 Data sources and original computational design . . . 13
- 3.2 Floating-point conventions . . . 13
- 3.3 Focused replication and figure generation . . . 13
- 3.4 Reproducibility and evidential status . . . 14

4 Results **15**
- 4.1 The power-of-two baseline . . . 15
- 4.2 Odd integer increments . . . 15
- 4.3 Dyadic and non-dyadic fractional inputs . . . 19
- 4.4 Comparison between binary32 and binary64 . . . 21

5 Discussion and Limitations **22**
- 5.1 Why a small ULP error can be a large relative error . . . 22
- 5.2 Binade crossings and the region below c . . . 22
- 5.3 Input conversion and computation order . . . 23
- 5.4 Connection with Fast2Sum and possible remedies . . . 23
- 5.5 What is proved, observed, and open . . . 23

6 Conclusions **24**

References **25**

Appendices

A Reproducibility and Source Notes **26**
A.1 Generating the figures 26
A.2 Source-material inventory 26
A.3 Source attribution and publication status 26
A.4 Assumptions checklist 27

List of Figures

2.1 Reconstructed binary64 ULP staircase for x^{10}. The dashed line marks $u = 1$, and the dotted line marks the $u = 2$ transition. 9
2.2 Theoretical normalized rounding error as a function of the increment measured in ULPs. Values at half-integer ties require the parity rule. 12

4.1 Power-of-two increment map on the 780-case integer grid. At a given x, smaller increments are absorbed; the narrow tie boundary separates preserved and absorbed results. 16
4.2 Continuous numerical curves for $c = 2^{-1},2^{-3},2^{-6},2^{-10}$ (binary64), corresponding to the supplied continuous Plotly experiments. The shaded strip identifies the parity-sensitive binade. 16
4.3 Odd-integer experiments ($c = 3,5,11,21$) reconstructed in English from the supplied family of Plotly curves. Dashed horizontal lines show the exact increment; dotted vertical lines mark the predicted complete-absorption boundaries. 17
4.4 Odd-integer relative error over a dense binary64 grid. The predicted complete-absorption boundary is superimposed on the numerical results. . 18
4.5 Dyadic versus non-dyadic fractional increments in binary64. The lower panels show absolute relative error on a logarithmic vertical scale, exposing rounding long before absorption. 20
4.6 Examples $c = 1/d$ for selected odd denominators d from the source's reciprocal-fraction Plotly experiment. The dashed line is the exact value $1/d$. 20
4.7 Predicted inexact ranges in the two precisions for representative increments. The bars visualize the formulas for x_1 and x_2; they are not uncertainty intervals. 21

List of Tables

4.1 Classification of 780 binary64 evaluations with $x \in \{2,\ldots,40\}$ and $c = 2^{-k}$ for $k \in \{1,\ldots,20\}$. Counts were independently regenerated. 15
4.2 Predicted first inexact and fully absorbed boundaries for selected positive odd integers, binary64. Values are rounded to two decimal places. 17
4.3 Model values for $c = 101$ in binary64, taken from the supplied study. The tie binade has two possible values. 18
4.4 Predicted inexact intervals for representative fractional increments. Inputs 0.1 and 1/3 denote their respective values after conversion to the indicated precision. 19
4.5 Relative errors $(f - c)/c$ reported in the source manuscript for binary64 fractions. 19

1 Introduction

1.1 Motivation

The mathematical identity

$$(x^{10} + c) - x^{10} = c \tag{1.1}$$

holds for every real x and c. Its evaluation in floating-point arithmetic, however, does not necessarily return c. The first addition may round away part or all of the increment, even when the subsequent subtraction is itself exact. As the magnitude of x^{10} grows, the gap between representable numbers grows in discrete steps. This makes Equation (1.1) a compact way to observe rounding, absorption, and cancellation-related loss of information [1, 2].

The central quantity is the *unit in the last place* (ULP). Unlike a fixed absolute error tolerance, the ULP changes with the exponent of the stored floating-point number. An increment that is easily represented near one magnitude may lie exactly halfway between two representable results at a larger magnitude, then become too small to change the larger operand at all. The behavior is therefore not accurately described by a single threshold in x for every possible increment c.

1.2 Research question and aims

This report addresses the question: *How do the magnitude and binary bit structure of a positive floating-point increment c determine the first rounding error, the intermediate output values, and complete absorption in $(x^{10} + c) - x^{10}$?*

The specific aims are to describe a common model for four families of increments, connect its thresholds to IEEE 754 precision, present empirical results and figures from the supplied sources, and separate established mathematical statements from experimental observations and still-open questions. The expression is treated as a study of floating-point evaluation rather than as a proposal for a new numerical algorithm.

1.3 Contributions and provenance

The manuscript by Rocha dated 4 November 2026 provides a piecewise nearest-grid model, formulas for the two transition thresholds, and results for 1,611 increments in each of two precisions [3]. The accompanying extended draft develops proofs, reviews related error-free transformations, and identifies qualifications to the binade-wise description. A

separate seven-page experimental report and the `ulp.ipynb` notebook supply plots and a focused enumeration of negative powers of two. Four supplied Plotly HTML charts visualize the power-of-two, odd-integer, and fractional cases.

This document reorganizes these materials into an English-language report while preserving the distinction between the published mathematical framework of floating-point rounding and the particular threshold notation used in the supplied project. The figures are regenerated in English from equivalent NumPy experiments; the plotting script is included. The full source files supplied by the author remain the primary evidence for the larger-scale experiments, and large-scale validation counts are reported as source claims rather than as experiments rerun for this report.

1.4 Scope and assumptions

The analysis principally considers finite, positive, normal operands in IEEE 754 `binary32` (precision $p = 24$) and `binary64` ($p = 53$) [4]. Operations round to nearest, with ties resolved toward an even least-significant significand bit. The power x^{10} is evaluated once and its stored result y is reused in the addition and subtraction. No fused evaluation, extended-precision intermediate, overflow, or underflow is assumed in the central argument. Most exact statements additionally require $y \geq c$ and require that the exact sum $y + c$ remain within the binade of y. These qualifications matter in interpreting early fractional results and samples close to powers of two.

1.5 Organization

Chapter 2 introduces floating-point representation and derives the nearest-grid rule and thresholds. Chapter 3 records the experimental setup and source datasets. Chapter 4 reports observations, numeric tables, and visualizations. Chapter 5 discusses exceptions, error-free transformations, and practical consequences; Chapter 6 concludes. Appendix A supplies reproducibility instructions and notes about the source archives.

2 Theoretical Framework

2.1 Binary floating-point spacing

A normal binary floating-point number of precision p can be written

$$y = M\,2^{E-p+1}, \qquad 2^{p-1} \le M < 2^{p}, \quad M \in \mathbb{Z}, \tag{2.1}$$

where $E = \lfloor \log_2 y \rfloor$ is the binade exponent. If $y \in [2^{E},2^{E+1})$, the upward spacing throughout the interior of that binade is

$$u = \mathrm{ulp}(y) = 2^{E-p+1}. \tag{2.2}$$

The spacing is constant within each binade and doubles at the next power of two. For binary64, the transition from $u = 1$ to $u = 2$ occurs at $y = 2^{53}$; when $y \approx x^{10}$, this corresponds to $x \approx 2^{53/10} \approx 39.3966$. In binary32, the analogous transition is at $x \approx 2^{24/10} \approx 5.2780$.

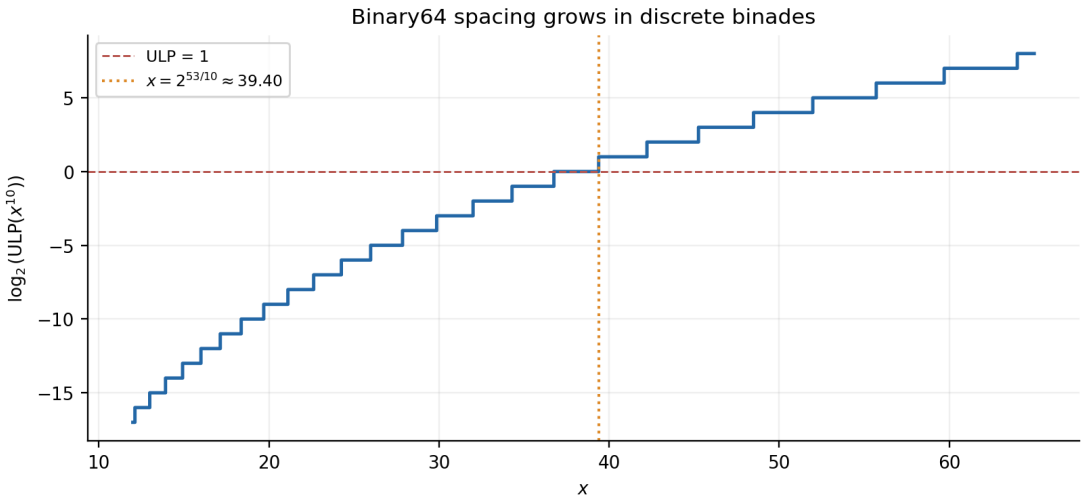


Figure 2.1: Reconstructed binary64 ULP staircase for x^{10}. The dashed line marks $u = 1$, and the dotted line marks the $u = 2$ transition.

2.2 The computed function and exact final subtraction

Let

$$y = \mathrm{fl}(x^{10}), \qquad s = \mathrm{fl}(y + c), \qquad f(x; c) = \mathrm{fl}(s - y). \tag{2.3}$$

A central fact is that the subtraction in Equation (2.3) need not introduce an additional error. Under $y \geq c > 0$ and normal finite arithmetic, s is sufficiently close to y for the subtraction to be exact by Sterbenz's lemma (which states that $a - b$ is exactly representable when two floating-point operands satisfy $b/2 \leq a \leq 2b$ under the usual exponent-range hypotheses) [5, 2]. Thus $f = s - y$ and the local error arises from the rounded addition.

Write the positive stored increment in lowest binary terms as

$$c = m2^{\nu}, \qquad m \text{ a positive odd integer}, \quad \lambda = \lfloor \log_2 c \rfloor. \tag{2.4}$$

Here $\nu = \nu(c)$ is the exponent of the least significant nonzero bit in the exact stored value of c. This is meaningful even when the user typed a non-dyadic decimal: the stored binary32 or binary64 number is always dyadic.

2.3 Nearest-grid rule and ties

Proposition 2.1 (Binade-wise rounding model)**.** *Assume round-to-nearest, ties-to-even; $y \geq c > 0$; reuse of the same stored y; and $y + c < 2^{E+1}$, so the exact sum remains in the same binade. Set $u = 2^{E-p+1}$, $M = y/u \in \mathbb{Z}$, and $q = c/u$. Then*

$$f(x; c) = u\,k, \quad k = \begin{cases} q, & q \in \mathbb{Z}, \\ \lfloor q + \frac{1}{2} \rfloor, & q \notin \mathbb{Z} + \frac{1}{2}, \\ \textit{the integer in } \{\lfloor q \rfloor, \lceil q \rceil\} \textit{ for which } M + k \textit{ is even}, & q \in \mathbb{Z} + \frac{1}{2}. \end{cases} \tag{2.5}$$

Proof. The stored $y = Mu$ is a point on the binade's representable grid. Rounding $y + c = u(M + q)$ to nearest selects $s = u(M + k)$, with ties resolved by even $M + k$. Sterbenz's lemma makes the final subtraction exact, yielding $f = s - y = uk$. □

In particular, all ties depend on the parity of the stored significand M, not only on the magnitude of c. For example, at $x = 40$ in binary64, $y = 40^{10} = 10{,}485{,}760{,}000{,}000{,}000$ is exactly representable, $u = 2$, and M is even. Thus $c = 1$ gives $f = 0$, $c = 3$ gives $f = 4$, and $c = 5$ gives $f = 4$. A neighboring stored y with odd M can select the other side of a tie.

2.4 Thresholds and the width of the inexact region

Proposition 2.2 (Transition binades)**.** *Under the hypotheses of Proposition 2.1, let*

$$E_1 = p + \nu, \qquad E_2 = p + \lambda, \qquad L = E_2 - E_1 + 1 = \lambda - \nu + 1. \tag{2.6}$$

Below E_1, the grid resolves c exactly. At E_1, the increment lies exactly halfway between two grid points and the result alternates between $c - u/2$ and $c + u/2$ according to parity. In subsequent binades through E_2, the rounded increment is a binade-dependent multiple of u; above E_2 it is zero. Consequently, in terms of x the predicted transition interval is approximately

$$x_1 = 2^{(p+\nu)/10}, \qquad x_2 = 2^{(p+1+\lambda)/10}, \qquad \frac{x_2}{x_1} = 2^{L/10}. \tag{2.7}$$

These are thresholds in the exponent of the computed *y; their numerical x locations may shift slightly because $y = \mathrm{fl}(x^{10})$ and because of finite input sampling.*

Proof. If $u \le 2^\nu$, the power-of-two grid spacing divides $c = m2^\nu$, so c/u is an integer and Equation (2.5) returns c. The first larger spacing is $u = 2^{\nu+1}$, where $q = m/2$ is a half-integer. For still larger powers of two, q is neither integral nor halfway between consecutive integers. The rounded result becomes identically zero once $u > 2c$, i.e., for exponents $E > p + \lambda$. The number of potentially inexact binades is therefore $L = (p+\lambda) - (p+\nu) + 1$. The conversion from exponents of $y \approx x^{10}$ produces Equation (2.7). If c is itself a power of two, the final tie binade can yield either zero or $2c$; the fully absorbed regime starts in the following binade. □

The integer L is the binary bit length of the odd significand m. For $c = 2^j$, $L = 1$. For positive odd integers, $\nu = 0$ and $L = \lfloor \log_2 c \rfloor + 1$. For typical stored approximations to non-dyadic fractions, L is close to the full precision p.

2.5 Error measured in ULPs

Whenever Proposition 2.1 applies, the normalized error is

$$\varepsilon = \frac{f - c}{u} = k - q, \qquad |\varepsilon| \le \frac{1}{2}. \tag{2.8}$$

Thus the absolute error is bounded by half a ULP *of* y in the stated regime. The bound is not a guarantee of small *relative* error: $(f - c)/c$ can reach -1 under complete absorption, and can be appreciable even before that point.

2.6 Qualifications at binade borders and for $y < c$

At the upper edge of a binade, $y + c$ may cross into the next binade, where the spacing is $2u$. The fixed-u nearest-grid formula then requires modification. The supplied extended manuscript gives a bound of $|f - c| \le u$ for this crossing case when $y \ge c$, instead of the interior bound $u/2$ [6].

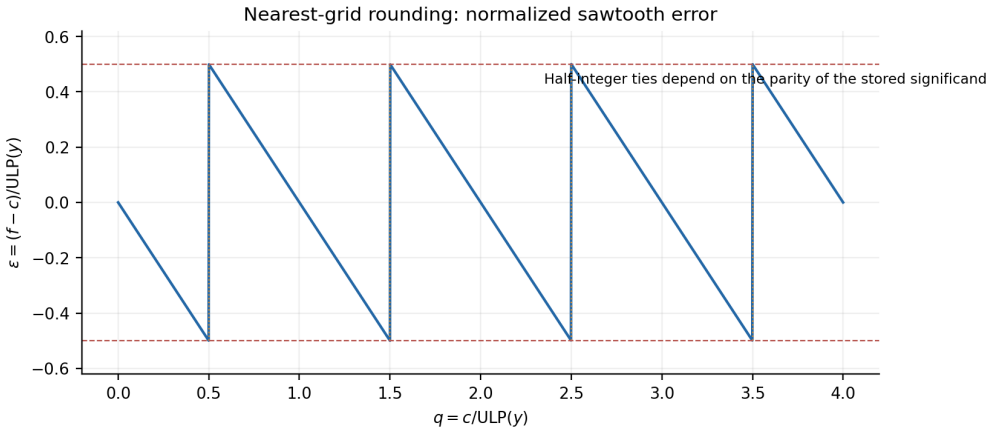


Figure 2.2: Theoretical normalized rounding error as a function of the increment measured in ULPs. Values at half-integer ties require the parity rule.

For $y < c$, the final subtraction need not satisfy Sterbenz's hypotheses, and the same manuscript provides a broader bound $|f - c| \leq 2\,\mathrm{ulp}(c)$ with a proof sketch. It reports the stronger empirical observation $|f - c| \leq \mathrm{ulp}(c)$ in tested samples, but does not claim to have proved that stronger statement. Neither phenomenon should be hidden by interpreting Equation (2.7) as a universal exact classification for every x.

3 Experimental Design and Reproduction

3.1 Data sources and original computational design

The principal inputs are the supplied eight-page research manuscript, its 42-page book-length draft, the seven-page preliminary report, the original `ulp.ipynb` notebook, four Plotly HTML exports, and the associated source-code archive. The preliminary report investigates $c = 2^{-k}$, $k = 1, \ldots, 20$, and c from two further families: odd integers and reciprocals of odd denominators. The manuscript and book draft extend these investigations to a systematic precision-dependent model.

The supplied validation code specifies three types of c, for each of binary64 and binary32: 1,001 odd integers from 1 to 2,001; 300 generated dyadic fractions; and 310 non-dyadic inputs consisting of ten named constants or fractions and 300 sampled real inputs. These categories total **1,611 increments per precision**. The stated random generator is `numpy.random.default_rng(42)`. The code samples 1,500–4,000 values of x per relevant binade, computes $y = x^{10}$, stores y once, and evaluates $(y + c) - y$. Results are grouped using the actual exponent of the computed y, with samples for which $y + c$ crosses a binade border handled separately.

3.2 Floating-point conventions

Arithmetic results are compared at the precision used for each experiment. The binary64 experiments use NumPy's `float64`, while binary32 experiments cast both x and c to `float32` before arithmetic to avoid unintentional promotion. Given a finite value y, the upward ULP in the numerical experiments is obtained from

$$\operatorname{ulp}(y) = \texttt{nextafter}(y, +\infty) - y. \tag{3.1}$$

For every sample the recorded quantities are the increment c, the actual y, the result f, its relative error $(f - c)/c$, and (where applicable) its ULP-scaled error $(f - c)/\operatorname{ulp}(y)$.

3.3 Focused replication and figure generation

A smaller deterministic check was independently executed during report preparation: x takes the 39 integers from 2 through 40, while k takes the 20 integers from 1 through 20, with $c = 2^{-k}$ and binary64 arithmetic. The 780 results are classified as preserved, absorbed,

tied downward, or tied upward. The resulting counts match the supplied preliminary report and are given in Table 4.1.

All included visualizations are generated by `scripts/generate\_figures.py`, which performs the arithmetic again with NumPy and saves static PNG plots with English axes and captions. The power-of-two map, continuous curves, odd-integer curves, and odd-reciprocal curves are counterparts of the four supplied interactive HTML exports. Additional plots recreate the ULP staircase, precision comparison, odd-integer error map, and the model's normalized rounding pattern described in the research manuscript. The original interactive HTML is not directly embedded in the PDF, since its interface labels and tooltips are in Portuguese and interactive JavaScript is not portable in a conventional LaTeX PDF.

3.4 Reproducibility and evidential status

The figures and the 780-case enumeration were regenerated locally. The 1,611-per-precision validation outcome, large-scale binade-border frequencies, and the original source tables are reported from the supplied study and were *not* rerun at their full sample sizes in this document. This distinction is important because the relevant conditions concern rounding behavior at extremely sparse boundary points. Source scripts are included in the accompanying materials for checking those experiments and for adapting them to new data.

4 Results

4.1 The power-of-two baseline

The simplest increment is $c = 2^{-k}$. The power-of-two grid implies only three possible numerical outcomes under the assumptions of Chapter 2: the original value c is preserved when $u \leq c$, the increment is absorbed when $u > 2c$, or a half-way case $u = 2c$ produces either 0 or $2c$ depending on parity. This is precisely the pattern observed by the source notebook.

Table 4.1: Classification of 780 binary64 evaluations with $x \in \{2, \ldots, 40\}$ and $c = 2^{-k}$ for $k \in \{1, \ldots, 20\}$. Counts were independently regenerated.

Outcome	Condition	Cases
Increment preserved, $f = c$	$c \geq u$	379
Increment absorbed, $f = 0$	$c < u/2$	371
Half-way, rounded downward	$c = u/2$	28
Half-way, rounded upward, $f = 2c$	$c = u/2$	2
Total		780

The 30 ties are not an implementation defect: both neighboring results are equally close to the exact sum, so ties-to-even selects according to the binary significand of y. Across a continuous x grid, different sampled significands exhibit both possible tie outcomes. The observed transition zone for $c = 2^{-1}$ is approximately $36.76 \leq x < 39.40$; for $c = 2^{-10}$ it is about $19.70 \leq x < 21.11$, as described in the supplied preliminary report.

4.2 Odd integer increments

For every positive odd integer c representable in the chosen precision, $\nu(c) = 0$. Therefore the onset of inexactness, when $y \geq c$ and binade crossing is excluded, does not depend on the *value* of c: the first affected binade is $E = p$. For binary64 this begins near $x_1 \approx 39.40$, while for binary32 it begins near $x_1 \approx 5.28$. The end of the inexact zone does depend on the size of c through $\lambda = \lfloor \log_2 c \rfloor$.

At the first inexact binade, $u = 2$ and every odd integer c lies halfway between two multiples of two. The output is therefore $c - 1$ or $c + 1$. For all later interior binades, $u = 4,8,16,\ldots$ and the computed increment changes by discrete jumps. In this regime f is not necessarily monotonic in x; it can move above and below the exact value as different low-order bits of c are discarded.

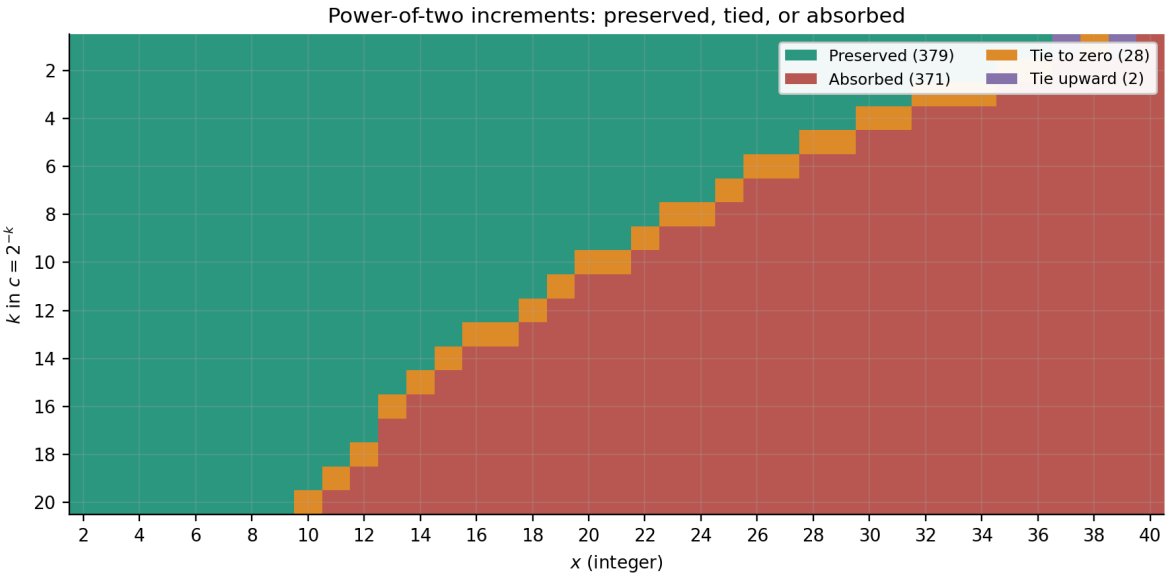


Figure 4.1: Power-of-two increment map on the 780-case integer grid. At a given x, smaller increments are absorbed; the narrow tie boundary separates preserved and absorbed results.

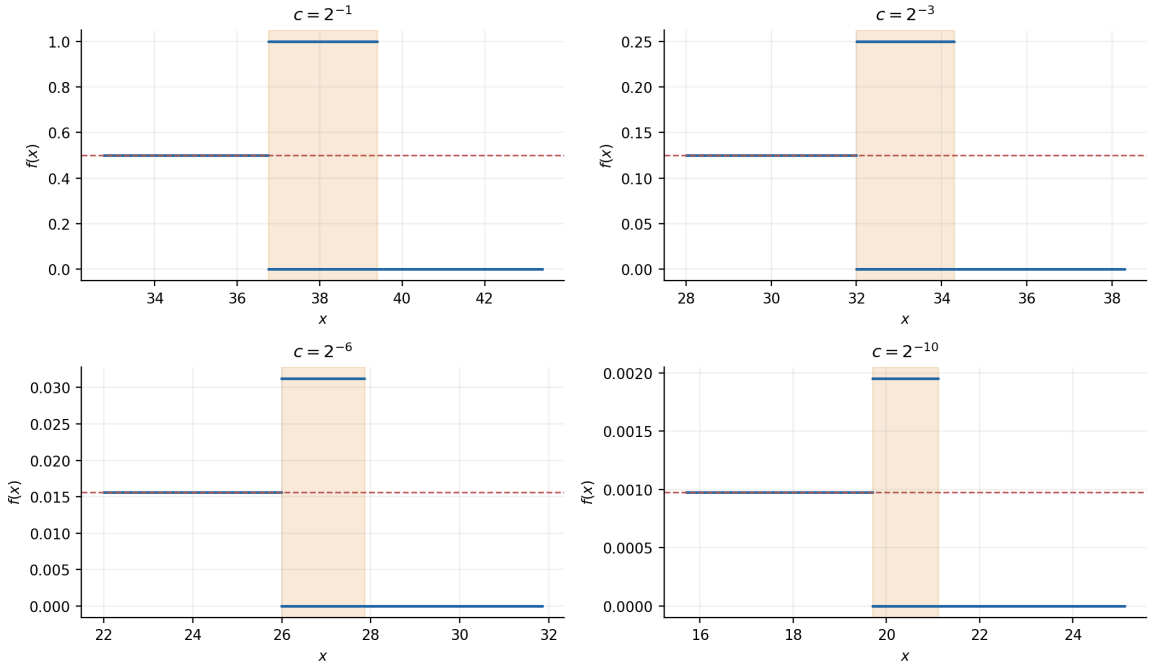


Figure 4.2: Continuous numerical curves for $c = 2^{-1}, 2^{-3}, 2^{-6}, 2^{-10}$ (binary64), corresponding to the supplied continuous Plotly experiments. The shaded strip identifies the parity-sensitive binade.

Table 4.2: Predicted first inexact and fully absorbed boundaries for selected positive odd integers, binary64. Values are rounded to two decimal places.

c	L (binades)	x_1	x_2
1	1	39.40	42.22
3	2	39.40	45.25
5 or 7	3	39.40	48.50
21	5	39.40	55.72
101	7	39.40	64.00
1001	10	39.40	78.79
65537	17	39.40	128.00

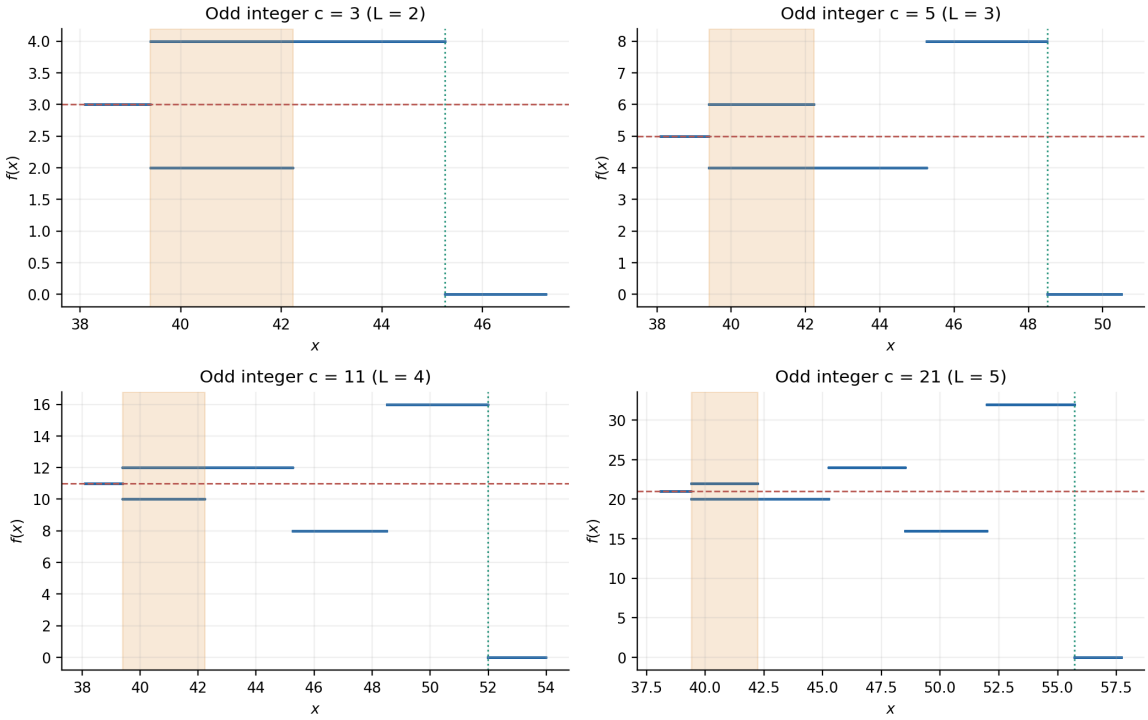


Figure 4.3: Odd-integer experiments ($c = 3,5,11,21$) reconstructed in English from the supplied family of Plotly curves. Dashed horizontal lines show the exact increment; dotted vertical lines mark the predicted complete-absorption boundaries.

For example, the manuscript presents $c = 101$ as a particularly informative case. The output is 100 or 102 at $u = 2$, then 100 at $u = 4$, 104 at $u = 8$, 96 at $u = 16$ and $u = 32$, 128 at $u = 64$ and $u = 128$, and 0 from $u = 256$ onward. The result 128 represents a relative error $(128 - 101)/101 \approx 26.7\%$, despite being less than half of the ULP of y in the relevant binade.

Table 4.3: Model values for $c = 101$ in binary64, taken from the supplied study. The tie binade has two possible values.

Exponent E	u	Computed f	Normalized error ε
$E < 53$	≤ 1	101	0
53	2	100 or 102	-0.5 or $+0.5$
54	4	100	-0.25
55	8	104	$+0.375$
56	16	96	-0.3125
57	32	96	-0.15625
58	64	128	$+0.421875$
59	128	128	$+0.2109375$
$E \geq 60$	≥ 256	0	≤ 0

The heatmap in Figure 4.4 summarizes odd values from 1 to 255. Horizontal bands arise because $x_2(c)$ changes only when c crosses a power of two, while the universal onset x_1 remains fixed. The drawn line gives the formula for the absorption threshold, not an independent fitted boundary.

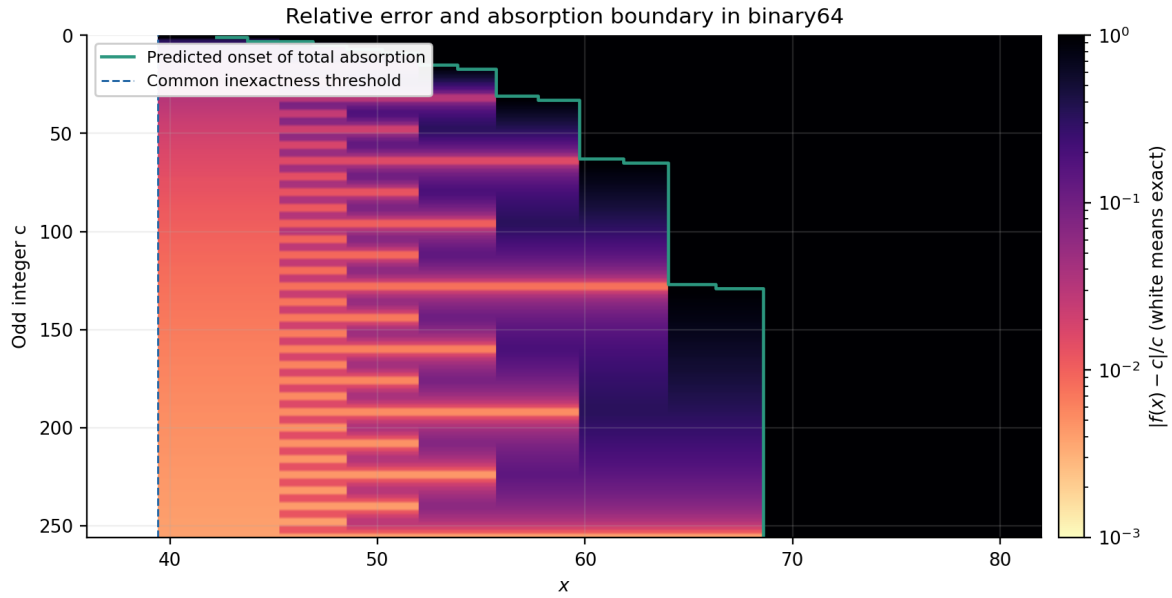


Figure 4.4: Odd-integer relative error over a dense binary64 grid. The predicted complete-absorption boundary is superimposed on the numerical results.

4.3 Dyadic and non-dyadic fractional inputs

Fractions of the form $m/2^j$ with odd m are dyadic. Their first inexact binade is shifted by j exponents compared with the same odd numerator: $\nu = -j$. Their width is still $L = \text{bitlength}(m)$. For example, $c = 3/4$ begins to round near $x_1 \approx 34.30$ and is fully absorbed from $x_2 \approx 39.40$; $c = 5/16$ begins near 29.86 and is fully absorbed from 36.76.

An input such as decimal 0.1 or mathematical 1/3 is not dyadic. The value actually stored in a binary computer is a finite dyadic approximation and generally has many nonzero significand bits. For binary64, the stored value nearest decimal 0.1 is $3602879701896397/2^{55}$, giving $\nu = -55$ and $L = 52$. For the stored binary64 approximation to 1/3, $L = 53$. These large bit lengths imply a much longer region of affected binades than the power-of-two case.

Table 4.4: Predicted inexact intervals for representative fractional increments. Inputs 0.1 and 1/3 denote their respective values after conversion to the indicated precision.

	binary64			binary32		
c	x_1	x_2	L	x_1	x_2	L
1/2	36.76	39.40	1	4.92	5.28	1
3/4	34.30	39.40	2	4.59	5.28	2
5/16	29.86	36.76	3	4.00	4.92	3
0.1	0.87	32.00	52	0.81	4.29	24
1/3	0.93	36.76	53	0.93	4.92	24

The supplied paper also reports relative errors for the stored approximations to 0.1 and 1/3 at selected binary64 inputs. Table 4.5 reproduces these source measurements, with the convention that $(f - c)/c$ is measured relative to the *stored* input c, not the ideal decimal constant. The relative error may be of order 10^{-9} at $x = 5$, yet exceed 10^{-1} by $x = 30$.

Table 4.5: Relative errors $(f - c)/c$ reported in the source manuscript for binary64 fractions.

x	$c = 0.1$	$c = 1/3$
1	$+8.3 \times 10^{-16}$	-1.7×10^{-16}
2	-9.1×10^{-13}	-2.3×10^{-13}
5	-3.7×10^{-9}	$+1.9 \times 10^{-9}$
10	$+3.8 \times 10^{-6}$	$+1.9 \times 10^{-6}$
20	-3.9×10^{-3}	$+2.0 \times 10^{-3}$
30	$+0.25$	$+0.125$
32.5	-1	-0.25

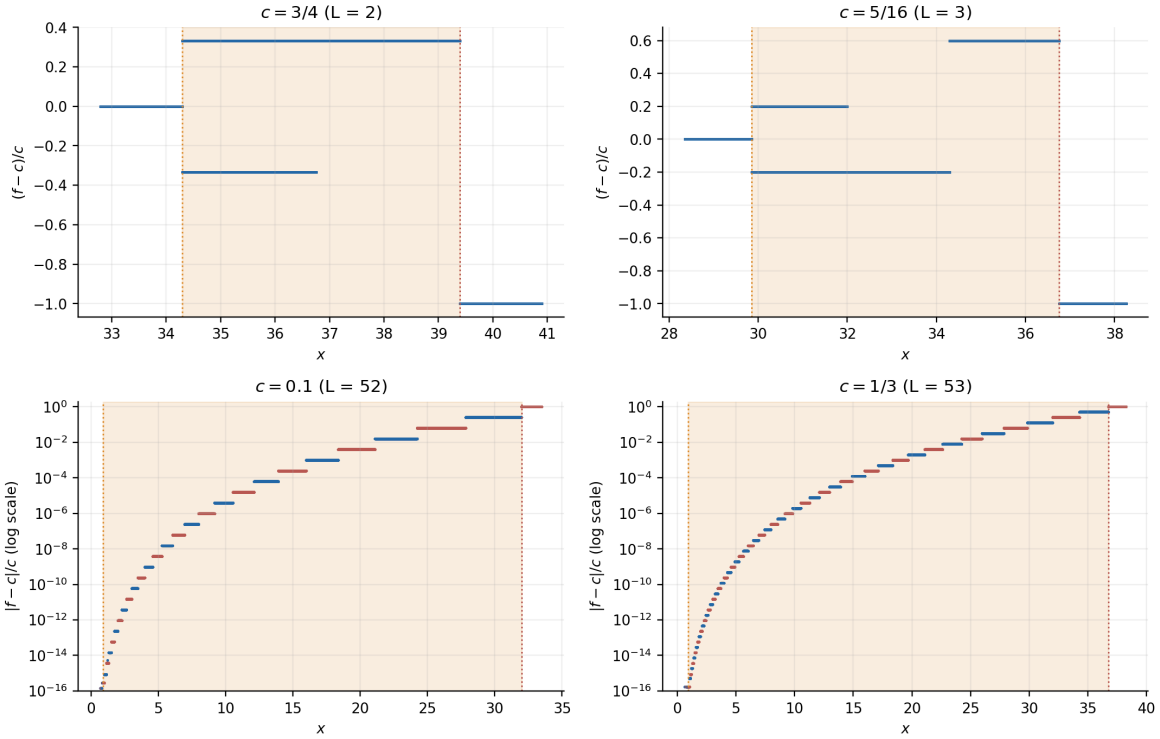


Figure 4.5: Dyadic versus non-dyadic fractional increments in binary64. The lower panels show absolute relative error on a logarithmic vertical scale, exposing rounding long before absorption.

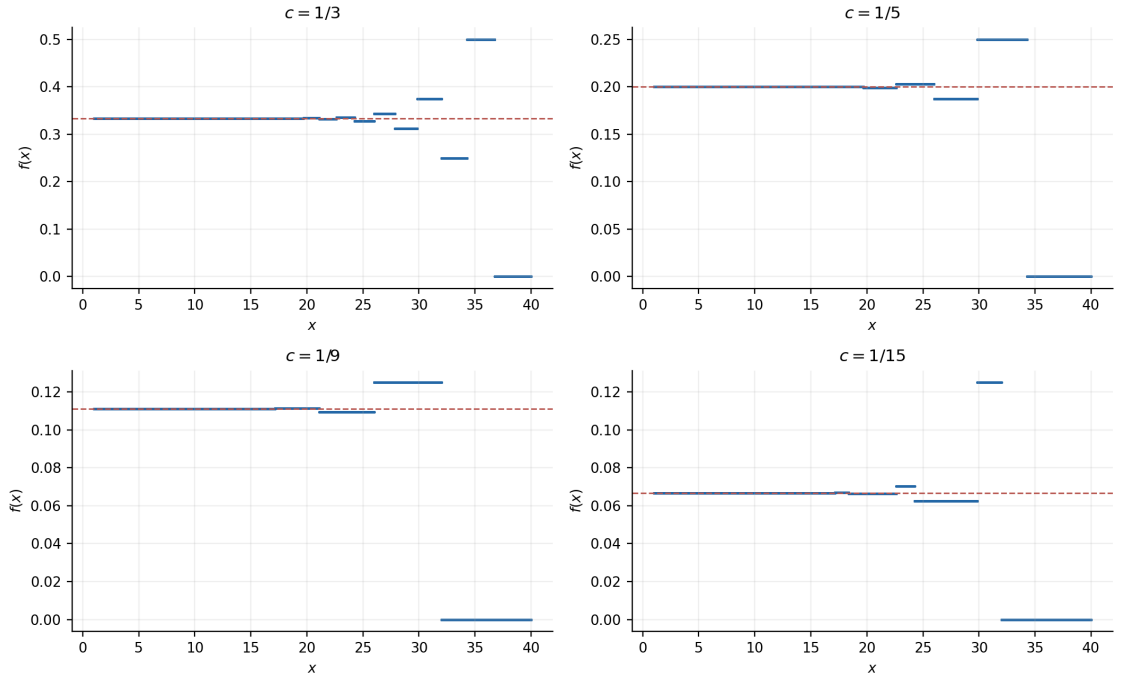


Figure 4.6: Examples $c = 1/d$ for selected odd denominators d from the source's reciprocal-fraction Plotly experiment. The dashed line is the exact value $1/d$.

4.4 Comparison between binary32 and binary64

The same model applies to both IEEE 754 formats after replacing the precision p. For odd integers, the first affected x decreases from approximately 39.40 in binary64 to 5.28 in binary32. The number L of affected binades is unchanged for an exactly representable c shared by both formats. For common non-dyadic literals, on the other hand, the binary32 approximation typically carries only about 24 relevant bits rather than about 53, and its predicted inexact interval is correspondingly narrower in log-space.

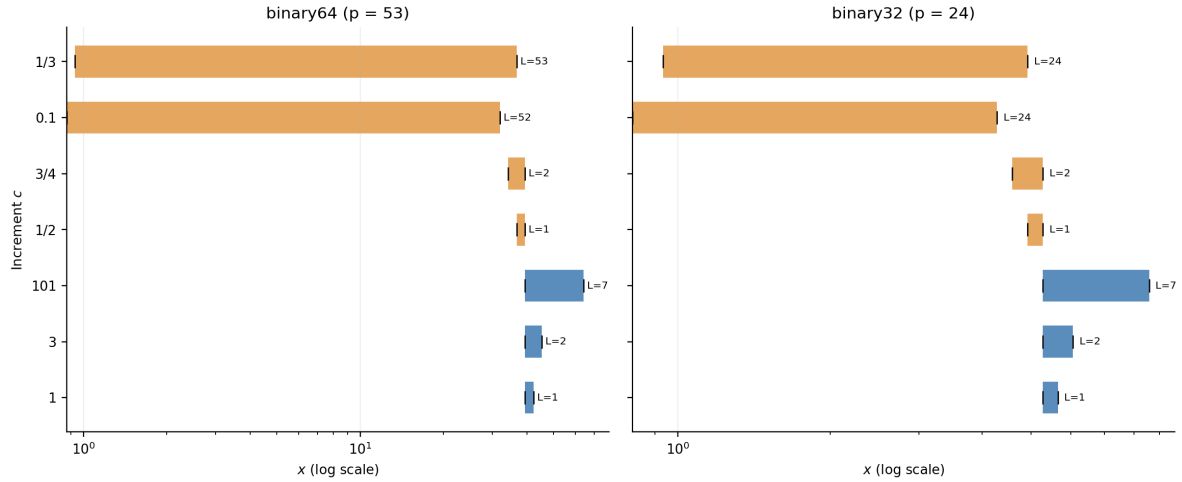


Figure 4.7: Predicted inexact ranges in the two precisions for representative increments. The bars visualize the formulas for x_1 and x_2; they are not uncertainty intervals.

The source study reports that the nearest-grid model matched its 1,611 tested increments in each precision after accounting for samples outside the model's domain (including binade crossings). These are reported outcomes from the original validation scripts. A particularly useful practical warning is that an input c may itself change when converted to binary32; comparisons must therefore use the *represented* c in each format rather than assume that the same exact real increment is available to both.

5 Discussion and Limitations

5.1 Why a small ULP error can be a large relative error

The nearest-grid result contains an important numerical distinction. Equation (2.8) limits the addition's rounding error to $u/2$ in the interior of a binade, but the ULP u belongs to the large quantity $y \approx x^{10}$, not to the small increment c. As x increases, u doubles every time x grows by a factor $2^{1/10} \approx 1.072$. With c fixed, an error small on the scale of y may be larger than the quantity one is trying to recover. Complete absorption occurs when the stored sum is identical to y, so the final subtraction gives exactly zero.

A subtle issue of terminology is that this expression involves both *absorption during addition* and *subtraction of nearby values*. The observed loss can occur even though the subtraction itself is exact. It is therefore more precise to attribute the loss in the model regime to rounding the addition, not to claim that Sterbenz's exact subtraction creates the error. This follows the error-analysis perspective of standard floating-point treatments [1, 2].

5.2 Binade crossings and the region below c

The main proposition assumes $y + c$ is smaller than the next power of two. For the small fraction of inputs very close to that boundary, the sum lands in a binade with twice the spacing; a one-ULP-of-y error bound replaces the interior half-ULP bound. The extended source draft explicitly distinguishes a proved bound from a heuristic for the frequency of this effect. It reports no such crossings among 4×10^7 sampled binary64 cases for odd or dyadic increments in the noisy region, 82 occurrences among the corresponding 4×10^7 binary32 cases, and approximately 0.3% and 1% for non-dyadic increments in binary64 and binary32, respectively. These reported frequencies are sampling results, not probabilities deduced exactly from the model.

At small x when $y < c$, Sterbenz's criterion may fail. The draft bounds the deviation by $2\,\mathrm{ulp}(c)$ and reports smaller deviations (at most one ULP of c) in the observed samples. Consequently, the familiar summary "exact, then noisy, then zero" should be read as a statement about the regular regime $y \geq c$ away from binade crossings; it is not an unconditional theorem for the entire positive real line.

5.3 Input conversion and computation order

Decimal literals such as `0.1` introduce a separate representation error before the experiment starts. In this report, $f - c$ means an error relative to the stored, exactly specified binary value of c. The error relative to the ideal real input is the sum of this conversion effect and the subsequent arithmetic effect. Likewise, evaluating $y = x^{10}$ with `pow`, repeated multiplication, or exponentiation by squaring can produce slightly different stored values. The nearest-grid reasoning applies to each resulting y, but the x positions of transitions may move. The submitted sources do not provide a controlled comparative benchmark of all these evaluation paths; none is invented here.

5.4 Connection with Fast2Sum and possible remedies

The intermediate result in Fast2Sum has the form

$$s = \mathrm{fl}(y + c), \qquad z = \mathrm{fl}(s - y), \qquad t = \mathrm{fl}(c - z). \tag{5.1}$$

Our computed function is exactly z under the same operation order. Error-free transformations such as Fast2Sum and TwoSum use additional operations to represent the residual of a floating-point addition, rather than discard it [7, 8]. When the algorithm's hypotheses hold, the pair (s,t) represents the exact sum $y + c$. This connection is already present in classical numerical-arithmetic literature; the supplied study's contribution is to express the intermediate z through $\nu(c)$, λ, and the x-thresholds.

For this *specific* mathematical identity, the best remedy is algebraic: compute c directly, without forming a large intermediate sum and subtracting it back. In less trivial computations, avoiding unnecessary cancellation, retaining a rounding residual when appropriate, and understanding compiler evaluation order are the more general lessons. A comparison of compensated summation methods is suggested by the source draft, but no new experimental claims about those methods are made here.

5.5 What is proved, observed, and open

The underlying proof rests on representable-grid rounding and the exactness of sufficiently close subtraction. Within the explicitly stated domain, Equations (2.5)–(2.7) are theoretical consequences; the half-ULP normalized error bound is also proved. The 780-case heatmap and plotted curves are local replications. The broader 1,611-input validation and the rare-crossing frequencies were reported by the supplied research sources.

Further work identified by the source authors includes the behavior for negative c, the exact distribution of binade-crossing events, precise bounds in the $y < c$ region, alternative ways of evaluating the power, fused or extended intermediate arithmetic, and extension to other precisions. The research draft also cautions against presenting the threshold notation as a demonstrated novelty without a more complete literature review, since the relevant exact-subtraction and error-free-transformation mechanisms are already established [6].

6 Conclusions

The numerical behavior of $(x^{10} + c) - x^{10}$ can be understood through the ULP of the reused stored power y and the low-order binary structure of the stored increment c. Away from binade crossings and with $y \geq c$, the output equals the increment rounded to the nearest multiple of $\mathrm{ulp}(y)$, with ties determined by the parity of the significand of y. This produces an initial exact regime, a parity-sensitive first inexact binade, discrete rounded plateaus, and complete absorption.

Writing $c = m2^{\nu}$ with odd m and $\lambda = \lfloor \log_2 c \rfloor$, the model predicts $x_1 \approx 2^{(p+\nu)/10}$ and $x_2 \approx 2^{(p+1+\lambda)/10}$, with $L = \lambda - \nu + 1$ potentially inexact binades. The distinction between one-bit increments and long-significand fractions explains why powers of two produce narrow transition bands whereas stored approximations to values such as 0.1 or 1/3 show error over a much broader range. In binary64, odd integers first become inexact near $x = 39.40$; in binary32 the corresponding point is near $x = 5.28$.

The supplied study reports a 1,611-increment validation in each precision, while the independently repeated 780-case power-of-two classification agrees with the preliminary report. Explicit assumptions and the exceptions at binade boundaries remain necessary. The example is pedagogically valuable because it exposes a key floating-point principle: an operation can be perfectly well rounded relative to its *large* operand and still completely destroy the information in its *small* operand.

References

1 GOLDBERG, D. What Every Computer Scientist Should Know About Floating-Point Arithmetic. *ACM Computing Surveys*, Association for Computing Machinery, v. 23, n. 1, p. 5–48, Mar. 1991. DOI: 10.1145/103162.103163.

2 HIGHAM, N. J. *Accuracy and Stability of Numerical Algorithms*. 2. ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2002. ISBN 9780898715217. DOI: 10.1137/1.9780898718027.

3 ROCHA, L. M. ULP in Floating-Point Arithmetic: The Function x to the Tenth plus c Minus x to the Tenth for Odd and Fractional Increments. Author-supplied manuscript, dated 4 November 2026; originally in Portuguese, 2026.

4 IEEE. *IEEE Standard for Floating-Point Arithmetic*. 2019. DOI: 10.1109/IEEESTD.2019.8766229.

5 STERBENZ, P. H. *Floating-Point Computation*. Prentice-Hall, 1974.

6 ROCHA, L. M. Cancellation, Absorption, and ULP in Floating-Point Arithmetic: A Study of the Expression (x to the n plus c) Minus x to the n in Binary32 and Binary64. Author-supplied extended draft, dated 4 November 2026; originally in Portuguese, 2026.

7 DEKKER, T. J. A Floating-Point Technique for Extending the Available Precision. *Numerische Mathematik*, v. 18, p. 224–242, 1971.

8 KNUTH, D. E. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. 3. ed.: Addison-Wesley, 1997.

Appendix A
Reproducibility and Source Notes

A.1 Generating the figures

From the directory containing `func\_nummet\_report.tex`, run:

```
python -m pip install numpy matplotlib
python scripts/generate_figures.py
latexmk -pdf -interaction=nonstopmode func_nummet_report.tex
```

The plotting script regenerates all nine English-language PNG files in `figs/`. The original experiments represented in the supplied source documents used NumPy and, for the interactive charts, Plotly. An English-commented adaptation of the source validation code is available at `scripts/source_validation/validate.py`; its full run is considerably more demanding than figure generation.

A.2 Source-material inventory

The supplied inputs comprised one LaTeX template archive; an extended Portuguese-language manuscript and corresponding source archive; an eight-page research article; a seven-page preliminary experimental report; a Jupyter notebook; and four interactive Plotly HTML visualizations (power-of-two grid, continuous power-of-two curves, odd integers, and odd reciprocals). The current document is an English synthesis of those supplied materials; it is not a verified independent replication of every experiment in the manuscript.

A.3 Source attribution and publication status

The supplied main article and book draft identify Lucas Martins Rocha as their author, and this report retains that attribution. The notebook originally extends material attributed in the supplied research manuscript to E. R. L. D. Ribas, `numError01`, under an MIT license. The template did not provide a verified supervisor or institutional affiliation for the author of the new report. Accordingly, no supervisor was inferred. The provided class and cover design are retained as a layout template; any institution-specific branding should be confirmed before external publication.

A.4 Assumptions checklist

The theoretical results should be applied only after checking: (i) finite positive stored c; (ii) normal finite values and no exception-producing overflow/underflow; (iii) round-to-nearest, ties-to-even; (iv) a single stored value of y reused; (v) no fused or extended-precision rearrangement; and, for the nearest-grid formula as stated, (vi) $y \geq c$ and $y + c < 2^{E+1}$. If these conditions are not met, the source manuscript analyzes some cases separately and identifies others as open.